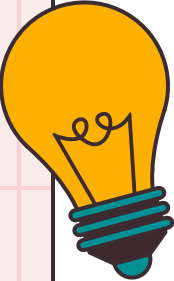




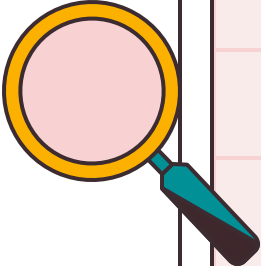
Isabella Llinas, Franklin
Gonzalez, Nicolas Bautista

Aprendizaje Maquina

Comparación de Modelos de Predicción de Precios de Vivienda

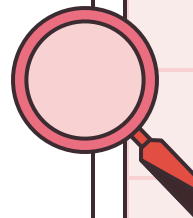


Análisis de Chicago y California
usando Decision Tree y KNN





Problema

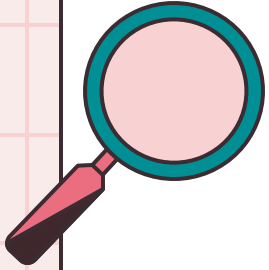


En el mercado inmobiliario, predecir el precio de las viviendas con precisión es fundamental para compradores, vendedores e inversionistas. Sin embargo, diferentes regiones tienen características particulares que pueden afectar la efectividad de los modelos de predicción. ¿Qué modelo de aprendizaje automático predice mejor los precios de vivienda en mercados en California?

California:

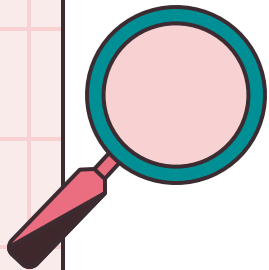
- Mercado altamente regulado con fuerte impacto de la oferta y demanda.
- Tendencia a precios elevados debido a la escasez de vivienda en áreas metropolitanas.
- Puede mostrar patrones más homogéneos en comparación con Chicago.

Dataset



Criterio	Explicación	Puntaje (1-5)
Tamaño del dataset	¿Tiene suficientes registros (mínimo 10,000 filas si es posible)?	5
Variables útiles	¿Incluye las variables clave para el problema?	5
Calidad de los datos	¿Hay datos faltantes, valores atípicos, errores?	4
Accesibilidad	¿Es un dataset público y descargable?	5
Formato y estructura	¿Está en CSV, JSON, o un formato fácil de procesar?	5

Metodología



1. Preparación de los datos:

- Limpieza de datos:
- Selección de variables:
- División de datos: (80%) (20%).

2. Modelos entrenados:

- **Árbol de Decisión (Decision Tree):**
- **K-Nearest Neighbors (KNN):**

3. Métricas de evaluación:

- **MAE (Error Absoluto Medio):**
Promedio de los errores absolutos entre los valores reales y predichos.
- **MSE (Error Cuadrático Medio):**
Penaliza los errores grandes al elevarlos al cuadrado.
- **RMSE (Raíz del Error Cuadrático Medio):** Evalúa la dispersión de los errores y permite comparar modelos con diferentes escalas.



Aplicacion de KNN y decision tree



01



Exploracion de datos California

```
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 20640 entries, 0 to 20639
```

```
Data columns (total 10 columns):
```

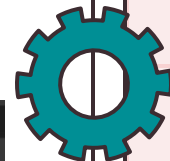
#	Column	Non-Null Count	Dtype
0	longitude	20640 non-null	float64
1	latitude	20640 non-null	float64
2	housing_median_age	20640 non-null	float64
3	total_rooms	20640 non-null	float64
4	total_bedrooms	20433 non-null	float64
5	population	20640 non-null	float64
6	households	20640 non-null	float64
7	median_income	20640 non-null	float64
8	median_house_value	20640 non-null	float64
9	ocean_proximity	20640 non-null	object

```
dtypes: float64(9), object(1)
```

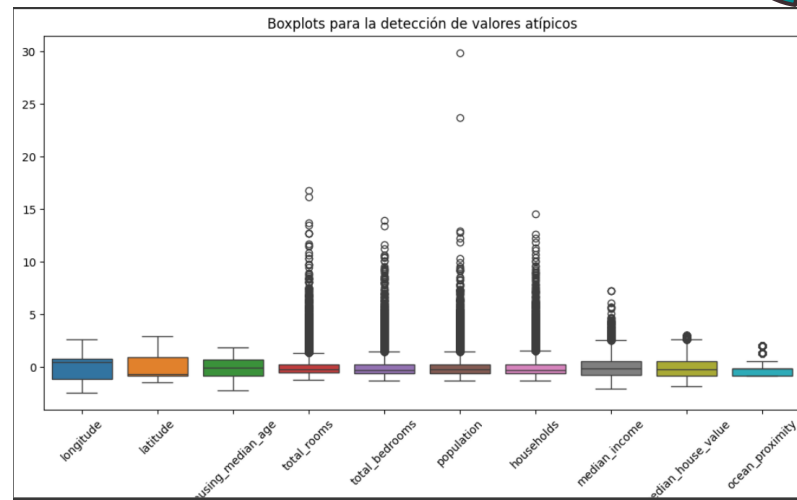
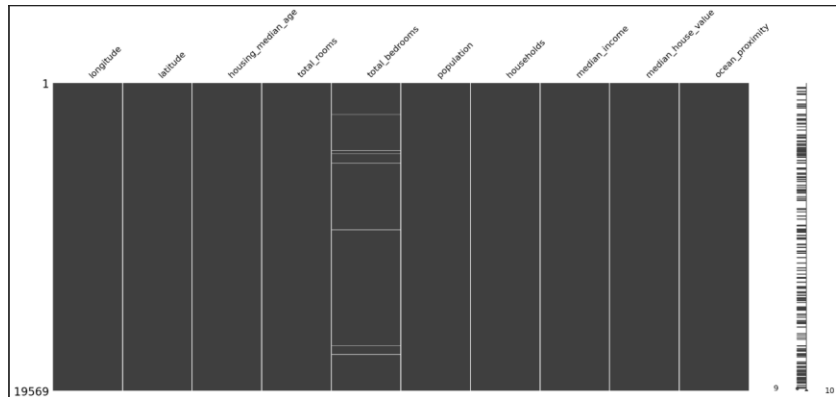
```
memory usage: 1.6+ MB
```

```
df2.describe()
```

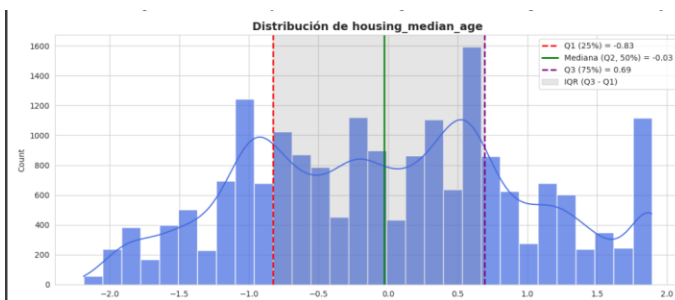
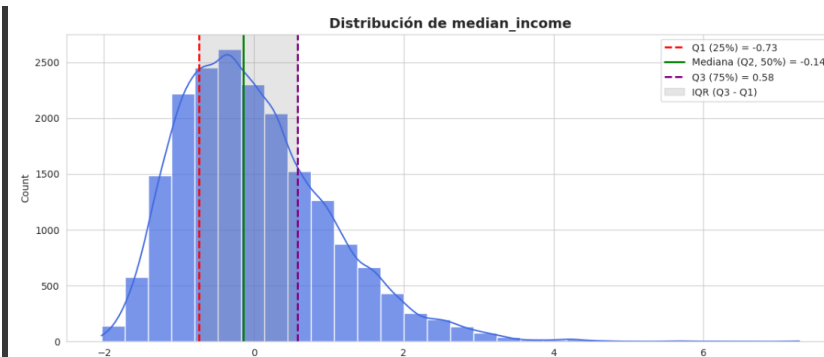
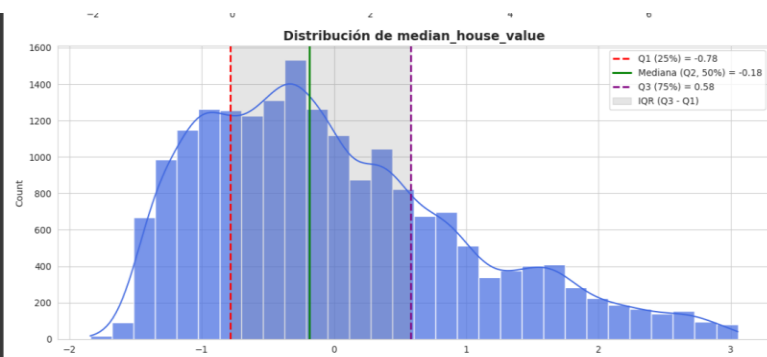
	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	206855.816909
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	115395.615874
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	14999.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	119600.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	179700.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	264725.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	500001.000000



Exploracion de datos California



Exploracion de datos California



SelectKBest with f_regression (k=9)

```
1 X = california.drop('median_house_value', axis=1)
2 y = california['median_house_value']
3
4 selector = SelectKBest(f_regression, k=9)
5 selector.fit_transform(X, y)
6 selectKBestColumns = X.columns[selector.get_support()]
7
8 X = california[selectKBestColumns]
9 y = california['median_house_value']
10
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
12
13 selectKBestColumns
```

```
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'ocean_proximity'],
      dtype='object')
```

Limpieza y seleccion de train y test

```
california.isnull().sum()
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	200
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

```
dtype: int64
```

```
5] columnas = ['housing_median_age', 'total_rooms', 'total_bedrooms', 'population', 'households', 'median_house_value']  
for i in columnas:  
    california[i]=california[i].astype(int)
```

```
california.dropna(inplace=True)  
california.isnull().sum()
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	0
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

Aplicacion de KNN

```
knn = KNeighborsRegressor(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print('Mean Absolute Error:', mean_absolute_error(y_test, y_pred))
print('Mean Squared Error:', mean_squared_error(y_test, y_pred))
print('R2 Score:', r2_score(y_test, y_pred))
```

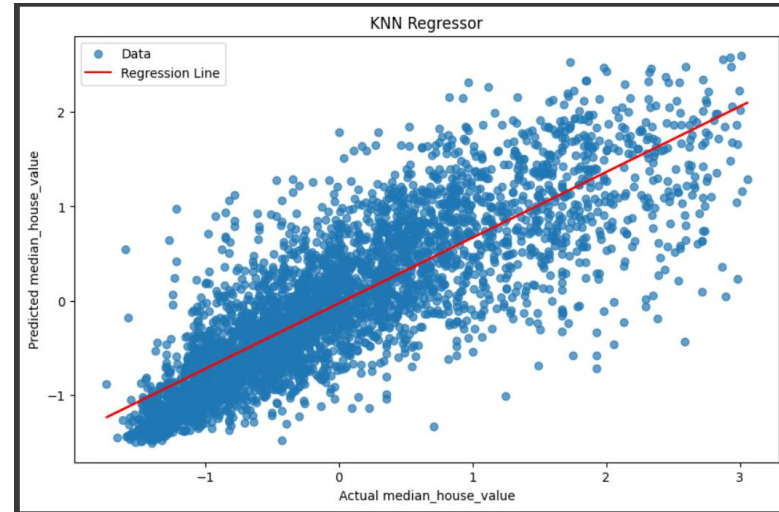
↕

```
Mean Absolute Error: 0.4019106912069074
Mean Squared Error: 0.32114451574760816
R2 Score: 0.6839705243631364
```

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.7, label='Data')

# Regression line
coefficients = np.polyfit(y_test, y_pred, 1)
regression_line = np.poly1d(coefficients)
plt.plot(y_test, regression_line(y_test), color='red', label='Regression Line')

plt.xlabel('Actual median_house_value')
plt.ylabel('Predicted median_house_value')
plt.title('KNN Regressor')
plt.legend()
plt.show()
```



Aplicacion de Decision tree

```
dt = DecisionTreeRegressor()
dt.fit(X_train, y_train)
y_pred = dt.predict(X_test)

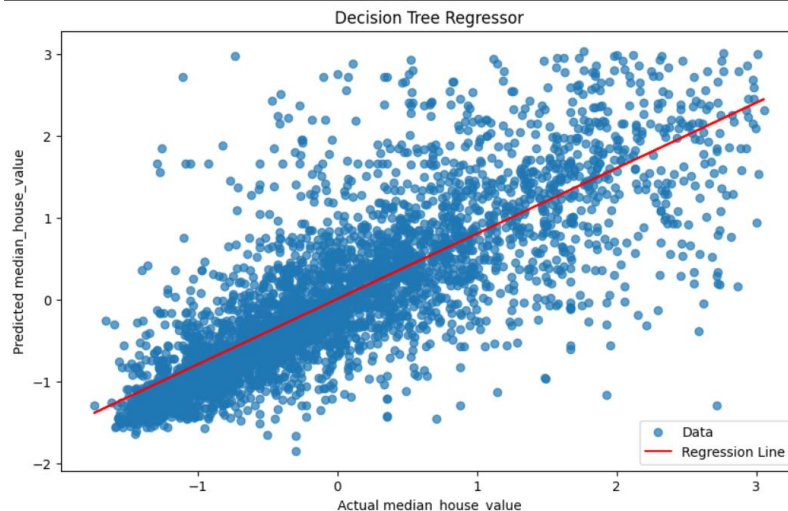
print('Mean Absolute Error:', mean_absolute_error(y_test, y_pred))
print('Mean Squared Error:', mean_squared_error(y_test, y_pred))
print('R2 Score:', r2_score(y_test, y_pred))

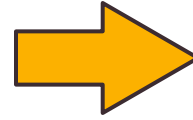
Mean Absolute Error: 0.4289189520398593
Mean Squared Error: 0.4143262376979198
R2 Score: 0.5922729574333598

plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.7, label='Data')

# Regression line
coefficients = np.polyfit(y_test, y_pred, 1)
regression_line = np.poly1d(coefficients)
plt.plot(y_test, regression_line(y_test), color='red', label='Regression Line')

plt.xlabel('Actual median_house_value')
plt.ylabel('Predicted median_house_value')
plt.title('Decision Tree Regressor')
plt.legend()
plt.show()
```





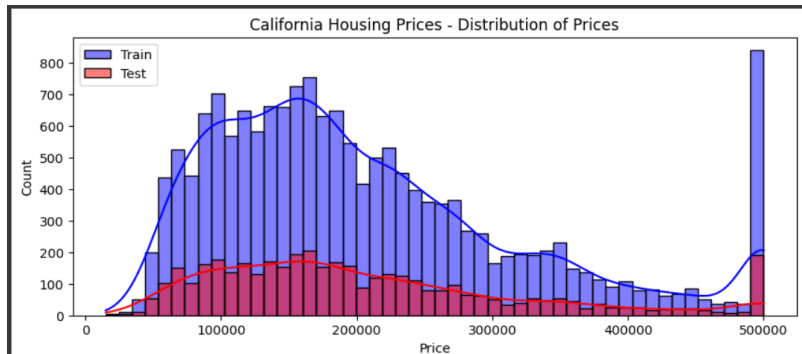
02

Resultados





Resultados



Distribución de precios:

Train (azul), Test (rojo).

Rango principal: \$100,000 - \$300,000.

Pico notable: Cerca de \$500,000.

KNN: Captura tendencia general, susceptible a ruido.

Árbol de Decisión: Predicciones escalonadas, posible sobreajuste, alta variabilidad.

KNN: Tiende a ofrecer predicciones más suaves y cercanas a la línea ideal, pero su rendimiento depende del número de vecinos seleccionados y puede verse afectado por datos ruidosos.

Árbol de Decisión: Puede sobreajustar los datos y generar predicciones menos generalizables, reflejado en la irregularidad de sus predicciones.

